

Spring Boot 监控数据可视化图文教程！

itmuch Web项目聚集地 1周前

点击上方“[Web项目聚集地](#)”，选择“置顶”公众号
重磅干货，第一时间送达

作者：itmuch | 来源：IT牧场

TIPS

本文基于Spring Boot 2.1.4，理论支持Spring Boot 2.x所有版本

众所周知，Spring Boot有个子项目Spring Boot Actuator，它为应用提供了强大的监控能力。从Spring Boot 2.0开始，Actuator将底层改为Micrometer，提供了更强、更灵活的监控能力。Micrometer是一个监控门面，可以类比成监控界的 [Slf4j](#)。

借助Micrometer，应用能够对接各种监控系统，例如：

- [AppOptics^{\[1\]}](#)
- [Atlas^{\[2\]}](#)
- [Datadog^{\[3\]}](#)
- [Dynatrace^{\[4\]}](#)
- [Elastic^{\[5\]}](#)
- [Ganglia^{\[6\]}](#)
- [Graphite^{\[7\]}](#)
- [Humio^{\[8\]}](#)
- [Influx^{\[9\]}](#)
- [JMX^{\[10\]}](#)
- [KairosDB^{\[11\]}](#)
- [New Relic^{\[12\]}](#)
- [Prometheus^{\[13\]}](#)
- [SignalFx^{\[14\]}](#)
- [Simple \(in-memory\)^{\[15\]}](#)

- [StatsD^{\[16\]}](#)
- [Wavefront^{\[17\]}](#)

下面演示如何对接 [Prometheus](#) ，并使用 [Grafana](#) 实现数据的可视化。

TIPS

童鞋们对Prometheus或Grafana不熟悉也没关系，本文是手把手文章，按步骤操作即可。

编码

编写代码

1 加依赖

```
1 <dependency>
2   <groupId>org.springframework.boot</groupId>
3   <artifactId>spring-boot-starter-actuator</artifactId>
4 </dependency>
5 <dependency>
6   <groupId>io.micrometer</groupId>
7   <artifactId>micrometer-registry-prometheus</artifactId>
8 </dependency>
```

这里，我们为应用引入了 [micrometer-registry-prometheus](#) ，事实上，你想对接上文列表中的哪款监控系统，就写啥。例如想对接 [Influx](#) ，则需添加依赖 [micrometer-registry-influx](#) 。

2 写配置

```
1 server:
2   port: 8080
3 spring:
4   application:
5     name: prometheus-test
6 management:
7   endpoints:
8     web:
```

```

9      exposure:
10      include: 'prometheus'
11  metrics:
12      tags:
13      application: ${spring.application.name}

```

如配置所示，指定应用名为 `prometheus-test`，并将 `Actuator` 的 `/actuator/prometheus` 端点暴露出来；
`management.metrics.tags.application=prometheus-test` 作用是为指标设置一个名为 `application="prometheus-test"` 的Tag，Tag是Prometheus提供的一种能力，从而实现更加灵活的筛选。

测试

1 启动应用

2 访问 `http://localhost:8080/actuator/prometheus` 可获得类似如下的结果：

```

1 # HELP jvm_memory_used_bytes The amount of used memory
2 # TYPE jvm_memory_used_bytes gauge
3 jvm_memory_used_bytes{application="prometheus-test",area="heap",id="PS Old Gen"} 1.0
4 jvm_memory_used_bytes{application="prometheus-test",area="nonheap",id="Metaspace"} 1.0
5 jvm_memory_used_bytes{application="prometheus-test",area="heap",id="PS Survivor Space"} 1.0
6 jvm_memory_used_bytes{application="prometheus-test",area="nonheap",id="Compressed Class Space"} 1.0
7 jvm_memory_used_bytes{application="prometheus-test",area="heap",id="PS Eden Space"} 1.0
8 jvm_memory_used_bytes{application="prometheus-test",area="nonheap",id="Code Cache"} 1.0
9 # HELP tomcat_global_received_bytes_total
10 # TYPE tomcat_global_received_bytes_total counter
11 tomcat_global_received_bytes_total{application="prometheus-test",name="http-nio-8080"} 1.0
12 # HELP jvm_threads_daemon_threads The current number of live daemon threads
13 # TYPE jvm_threads_daemon_threads gauge
14 jvm_threads_daemon_threads{application="prometheus-test",} 20.0
15 # HELP tomcat_sessions_alive_max_seconds
16 # TYPE tomcat_sessions_alive_max_seconds gauge
17 tomcat_sessions_alive_max_seconds{application="prometheus-test",} 0.0
18 # HELP jvm_buffer_memory_used_bytes An estimate of the memory that the Java VM has used in total for buffer memory
19 # TYPE jvm_buffer_memory_used_bytes gauge
20 jvm_buffer_memory_used_bytes{application="prometheus-test",id="mapped",} 0.0
21 jvm_buffer_memory_used_bytes{application="prometheus-test",id="direct",} 901.0
22 # HELP jvm_threads_states_threads The current number of threads having NEW state

```

```
23 # TYPE jvm_threads_states_threads gauge
24 jvm_threads_states_threads{application="prometheus-test",state="runnable",} 9.0
25 jvm_threads_states_threads{application="prometheus-test",state="new",} 0.0
26 jvm_threads_states_threads{application="prometheus-test",state="terminated",} 0.0
27 jvm_threads_states_threads{application="prometheus-test",state="blocked",} 0.0
28 jvm_threads_states_threads{application="prometheus-test",state="waiting",} 1.0
29 jvm_threads_states_threads{application="prometheus-test",state="timed-waiting",} 0.0
30 # HELP process_cpu_usage The "recent cpu usage" for the Java Virtual Machine
31 # TYPE process_cpu_usage gauge
32 process_cpu_usage{application="prometheus-test",} 0.0030590633504868434
33 # HELP logback_events_total Number of error level events that made it to the log
34 # TYPE logback_events_total counter
35 logback_events_total{application="prometheus-test",level="info",} 7.0
36 logback_events_total{application="prometheus-test",level="warn",} 0.0
37 logback_events_total{application="prometheus-test",level="trace",} 0.0
38 logback_events_total{application="prometheus-test",level="debug",} 0.0
39 logback_events_total{application="prometheus-test",level="error",} 0.0
40 # HELP tomcat_global_sent_bytes_total
41 # TYPE tomcat_global_sent_bytes_total counter
42 tomcat_global_sent_bytes_total{application="prometheus-test",name="http-nio-8080",} 0.0
43 # HELP process_files_max_files The maximum file descriptor count
44 # TYPE process_files_max_files gauge
45 process_files_max_files{application="prometheus-test",} 10240.0
46 # HELP tomcat_threads_busy_threads
47 # TYPE tomcat_threads_busy_threads gauge
48 tomcat_threads_busy_threads{application="prometheus-test",name="http-nio-8080",} 0.0
49 # HELP process_files_open_files The open file descriptor count
50 # TYPE process_files_open_files gauge
51 process_files_open_files{application="prometheus-test",} 101.0
52 # HELP tomcat_sessions_active_current_sessions
53 # TYPE tomcat_sessions_active_current_sessions gauge
54 tomcat_sessions_active_current_sessions{application="prometheus-test",} 0.0
55 # HELP jvm_classes_unloaded_classes_total The total number of classes unloaded since JVM
56 # TYPE jvm_classes_unloaded_classes_total counter
57 jvm_classes_unloaded_classes_total{application="prometheus-test",} 2.0
58 # HELP jvm_memory_committed_bytes The amount of memory in bytes that is committed
59 # TYPE jvm_memory_committed_bytes gauge
60 jvm_memory_committed_bytes{application="prometheus-test",area="heap",id="PS MemoryPool",} 1.0736e+08
61 jvm_memory_committed_bytes{application="prometheus-test",area="nonheap",id="PermGenSpace",} 1.0736e+08
62 jvm_memory_committed_bytes{application="prometheus-test",area="heap",id="PS MemoryPool",} 1.0736e+08
```

```
63 jvm_memory_committed_bytes{application="prometheus-test",area="nonheap",id="(
64 jvm_memory_committed_bytes{application="prometheus-test",area="heap",id="PS I
65 jvm_memory_committed_bytes{application="prometheus-test",area="nonheap",id="(
66 # HELP http_server_requests_seconds
67 # TYPE http_server_requests_seconds summary
68 http_server_requests_seconds_count{application="prometheus-test",exception="I
69 http_server_requests_seconds_sum{application="prometheus-test",exception="Noi
70 http_server_requests_seconds_count{application="prometheus-test",exception="I
71 http_server_requests_seconds_sum{application="prometheus-test",exception="Noi
72 # HELP http_server_requests_seconds_max
73 # TYPE http_server_requests_seconds_max gauge
74 http_server_requests_seconds_max{application="prometheus-test",exception="Noi
75 http_server_requests_seconds_max{application="prometheus-test",exception="Noi
76 # HELP tomcat_global_error_total
77 # TYPE tomcat_global_error_total counter
78 tomcat_global_error_total{application="prometheus-test",name="http-nio-8080".
79 # HELP jvm_gc_max_data_size_bytes Max size of old generation memory pool
80 # TYPE jvm_gc_max_data_size_bytes gauge
81 jvm_gc_max_data_size_bytes{application="prometheus-test",} 2.863661056E9
82 # HELP jvm_gc_live_data_size_bytes Size of old generation memory pool after :
83 # TYPE jvm_gc_live_data_size_bytes gauge
84 jvm_gc_live_data_size_bytes{application="prometheus-test",} 2.1193976E7
85 # HELP system_load_average_1m The sum of the number of runnable entities quee
86 # TYPE system_load_average_1m gauge
87 system_load_average_1m{application="prometheus-test",} 3.9423828125
88 # HELP process_uptime_seconds The uptime of the Java virtual machine
89 # TYPE process_uptime_seconds gauge
90 process_uptime_seconds{application="prometheus-test",} 173.424
91 # HELP tomcat_sessions_expired_sessions_total
92 # TYPE tomcat_sessions_expired_sessions_total counter
93 tomcat_sessions_expired_sessions_total{application="prometheus-test",} 0.0
94 # HELP jvm_gc_pause_seconds Time spent in GC pause
95 # TYPE jvm_gc_pause_seconds summary
96 jvm_gc_pause_seconds_count{action="end of minor GC",application="prometheus-t
97 jvm_gc_pause_seconds_sum{action="end of minor GC",application="prometheus-te
98 jvm_gc_pause_seconds_count{action="end of major GC",application="prometheus-t
99 jvm_gc_pause_seconds_sum{action="end of major GC",application="prometheus-te
100 # HELP jvm_gc_pause_seconds_max Time spent in GC pause
101 # TYPE jvm_gc_pause_seconds_max gauge
102 jvm_gc_pause_seconds_max{action="end of minor GC",application="prometheus-te
```

```
103 jvm_gc_pause_seconds_max{action="end of major GC",application="prometheus-test"} 1.06878E7
104 # HELP jvm_gc_memory_promoted_bytes_total Count of positive increases in the
105 # TYPE jvm_gc_memory_promoted_bytes_total counter
106 jvm_gc_memory_promoted_bytes_total{application="prometheus-test",} 1.06878E7
107 # HELP jvm_gc_memory_allocated_bytes_total Incremented for an increase in the
108 # TYPE jvm_gc_memory_allocated_bytes_total counter
109 jvm_gc_memory_allocated_bytes_total{application="prometheus-test",} 7.8713648
110 # HELP tomcat_global_request_seconds
111 # TYPE tomcat_global_request_seconds summary
112 tomcat_global_request_seconds_count{application="prometheus-test",name="http-nio-8080"} 1
113 tomcat_global_request_seconds_sum{application="prometheus-test",name="http-nio-8080"} 1.556017153576E9
114 # HELP jvm_threads_live_threads The current number of live threads including
115 # TYPE jvm_threads_live_threads gauge
116 jvm_threads_live_threads{application="prometheus-test",} 24.0
117 # HELP jvm_buffer_count_buffers An estimate of the number of buffers in the pool
118 # TYPE jvm_buffer_count_buffers gauge
119 jvm_buffer_count_buffers{application="prometheus-test",id="mapped",} 0.0
120 jvm_buffer_count_buffers{application="prometheus-test",id="direct",} 11.0
121 # HELP process_start_time_seconds Start time of the process since unix epoch
122 # TYPE process_start_time_seconds gauge
123 process_start_time_seconds{application="prometheus-test",} 1.556017153576E9
124 # HELP tomcat_threads_config_max_threads
125 # TYPE tomcat_threads_config_max_threads gauge
126 tomcat_threads_config_max_threads{application="prometheus-test",name="http-nio-8080"} 8.0
127 # HELP system_cpu_count The number of processors available to the Java virtual machine
128 # TYPE system_cpu_count gauge
129 system_cpu_count{application="prometheus-test",} 8.0
130 # HELP tomcat_threads_current_threads
131 # TYPE tomcat_threads_current_threads gauge
132 tomcat_threads_current_threads{application="prometheus-test",name="http-nio-8080"} 8.0
133 # HELP jvm_memory_max_bytes The maximum amount of memory in bytes that can be allocated
134 # TYPE jvm_memory_max_bytes gauge
135 jvm_memory_max_bytes{application="prometheus-test",area="heap",id="PS Old Gen"} 2.147483647E8
136 jvm_memory_max_bytes{application="prometheus-test",area="nonheap",id="Metaspace"} 2.147483647E8
137 jvm_memory_max_bytes{application="prometheus-test",area="heap",id="PS Survivor Space"} 2.147483647E8
138 jvm_memory_max_bytes{application="prometheus-test",area="nonheap",id="Compressed Class Space"} 2.147483647E8
139 jvm_memory_max_bytes{application="prometheus-test",area="heap",id="PS Eden Space"} 2.147483647E8
140 jvm_memory_max_bytes{application="prometheus-test",area="nonheap",id="Code Cache"} 2.147483647E8
141 # HELP jvm_threads_peak_threads The peak live thread count since the Java virtual machine started
142 # TYPE jvm_threads_peak_threads gauge
```

```

143 jvm_threads_peak_threads{application="prometheus-test",} 29.0
144 # HELP jvm_classes_loaded_classes The number of classes that are currently lo
145 # TYPE jvm_classes_loaded_classes gauge
146 jvm_classes_loaded_classes{application="prometheus-test",} 7936.0
147 # HELP tomcat_sessions_active_max_sessions
148 # TYPE tomcat_sessions_active_max_sessions gauge
149 tomcat_sessions_active_max_sessions{application="prometheus-test",} 0.0
150 # HELP tomcat_global_request_max_seconds
151 # TYPE tomcat_global_request_max_seconds gauge
152 tomcat_global_request_max_seconds{application="prometheus-test",name="http-n
153 # HELP tomcat_sessions_created_sessions_total
154 # TYPE tomcat_sessions_created_sessions_total counter
155 tomcat_sessions_created_sessions_total{application="prometheus-test",} 0.0
156 # HELP tomcat_sessions_rejected_sessions_total
157 # TYPE tomcat_sessions_rejected_sessions_total counter
158 tomcat_sessions_rejected_sessions_total{application="prometheus-test",} 0.0
159 # HELP jvm_buffer_total_capacity_bytes An estimate of the total capacity of t
160 # TYPE jvm_buffer_total_capacity_bytes gauge
161 jvm_buffer_total_capacity_bytes{application="prometheus-test",id="mapped",} 0
162 jvm_buffer_total_capacity_bytes{application="prometheus-test",id="direct",} 0
163 # HELP system_cpu_usage The "recent cpu usage" for the whole system
164 # TYPE system_cpu_usage gauge
165 system_cpu_usage{application="prometheus-test",} 0.2523364485981308

```

该端点返回的数据结构是Prometheus需要使用的。对照注释，相信大家能看懂每一项的含义。例如：

```

1 # HELP jvm_memory_used_bytes The amount of used memory
2 # TYPE jvm_memory_used_bytes gauge
3 jvm_memory_used_bytes{application="prometheus-test",area="heap",id="PS Old Gen"} 2.1193976E7

```

表示：**prometheus-test** 应用堆内存中的 **PS Old Gen** 区域占用的空间是 **2.1193976E7** 字节。

对接Prometheus

Prometheus是一款开源的监控 + 时序数据库 + 报警软件，由SoundCloud公司开发的。目前在CNCF基金会托管，并已成功孵化。

官方网站

- [Prometheus官方网站^{\[18\]}](#)
- [Prometheus GitHub^{\[19\]}](#)

安装Prometheus

TIPS

简单期间，本文使用Docker方式安装，其他方式安装请详见 [Prometheus安装说明^{\[20\]}](#)。

1 准备配置文件，命名为 `prometheus.yml`

```
1 scrape_configs:
2   # 任意写，建议英文，不要包含特殊字符
3   - job_name: 'spring'
4     # 多久采集一次数据
5     scrape_interval: 15s
6     # 采集时的超时时间
7     scrape_timeout: 10s
8     # 采集的路径是啥
9     metrics_path: '/actuator/prometheus'
10    # 采集服务的地址，设置成上面Spring Boot应用所在服务器的具体地址。
11    static_configs:
12      - targets: ['30.1.229.89:8080']
```

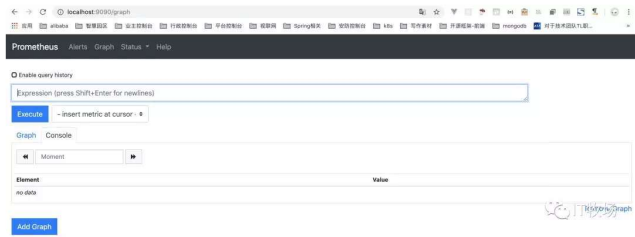
这样配置后，Prometheus服务器就会自动每隔15秒请求 <http://30.1.229.89:8080/actuator/prometheus> 。更多配置项详见 [Prometheus Configuration官方文档^{\[21\]}](#)

2 在 `prometheus.yml` 文件所在目录执行如下命令，即可启动Prometheus

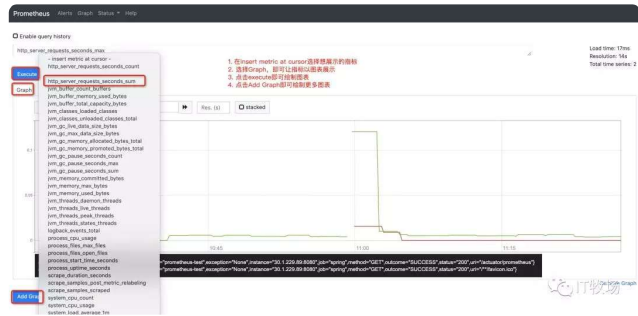
```
1 docker run -d -p 9090:9090 \
2   -v $(pwd)/prometheus.yml:/etc/prometheus/prometheus.yml \
3   prom/prometheus --config.file=/etc/prometheus/prometheus.yml
```

测试

1 访问 <http://localhost:9090> ，可看到类似如下的界面



2 点击 **Insert metric at cursor** ，即可选择监控指标；点击 **Graph** ，即可让指标以图表方式展示；点击 **Execute** 按钮，即可看到类似下图的结果：



Grafana可视化

至此，已经用Prometheus实现了监控数据的可视化，然而使用体验并不好。下面来用Grafana实现更友好、更贴近生产的监控可视化。

Grafana是一个开源的跨平台度量分析和可视化 + 告警工具。

官方网站

- [Grafana官方网站](#)^[22]
- [Grafana GitHub](#)^[23]

安装Grafana

TIPS

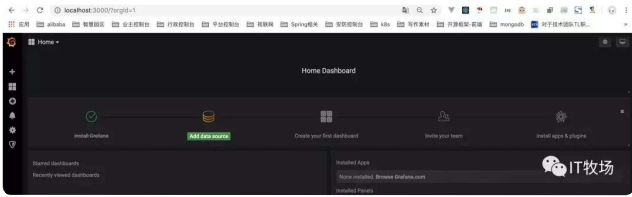
简单期间，本文使用Docker方式安装，其他方式安装请详见 [Grafana安装说明](#)^[24]。

启动

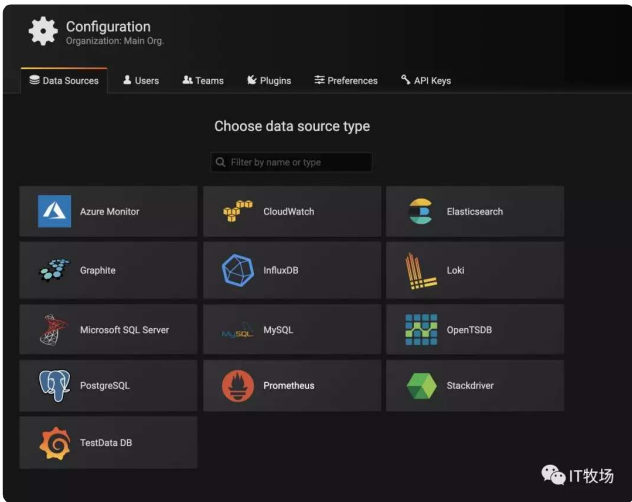
```
1 docker run -d -p 3000:3000 grafana/grafana
```

配置

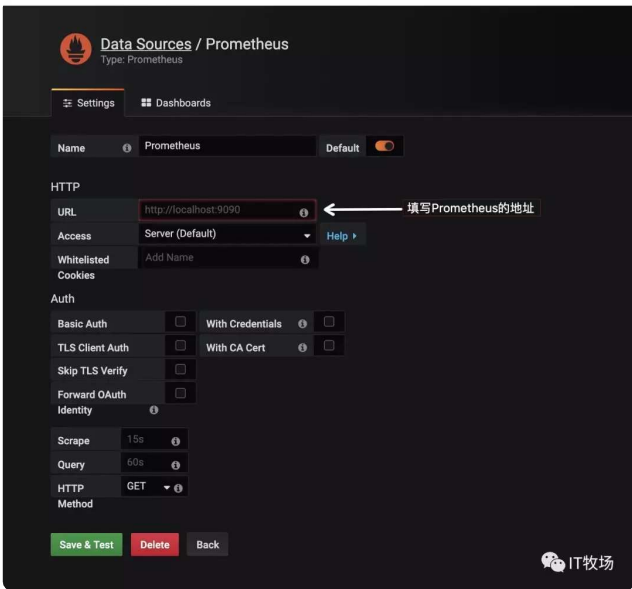
1 登录：访问 `http://localhost:3000/login` ，初始账号/密码为：`admin/admin` ，登录后，即可看到类似如下的界面



2 点击 `Add data source` ，即可看到类似如下的界面：



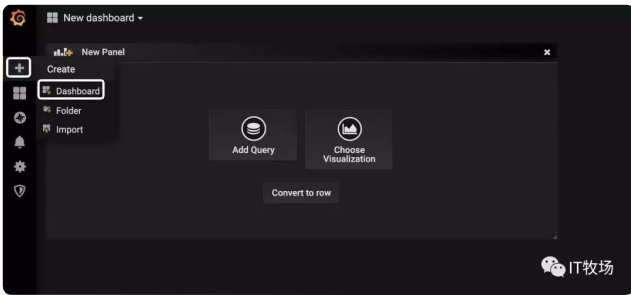
3 点击Prometheus，即可看到类似如下界面：



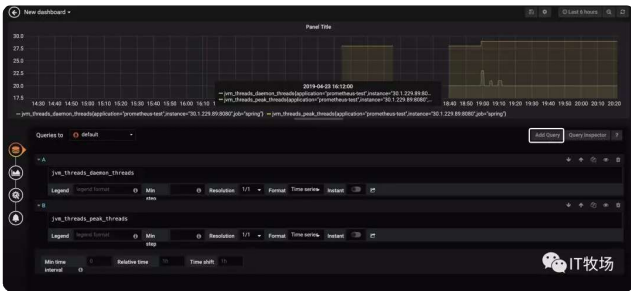
4 填写Prometheus的首页地址，并点击 `Save & Test`

创建监控Dashboard

1 点击导航栏上的 `+` 按钮，并点击Dashboard，将会看到类似如下的界面：



2 点击 **Add Query** ，即可看到类似如下的界面：



在白框标记的位置输入指标，指标的取值详见Spring Boot应用的 `/actuator/prometheus` 端点，例如 `jvm_memory_used_bytes` 、
`jvm_threads_states_threads` 、`jvm_threads_live_threads` 等，Grafana会给你较好的提示，并且支持较为复杂的计算，例如聚合、求和、平均等。如果想要绘制多个线条，可点击 **Add Query** 按钮，如上图所示，笔者为图表绘制了两条线，分别代表daemon以及peak线程。

3 点击下图的按钮，并填入Title，即可设置图表标题。



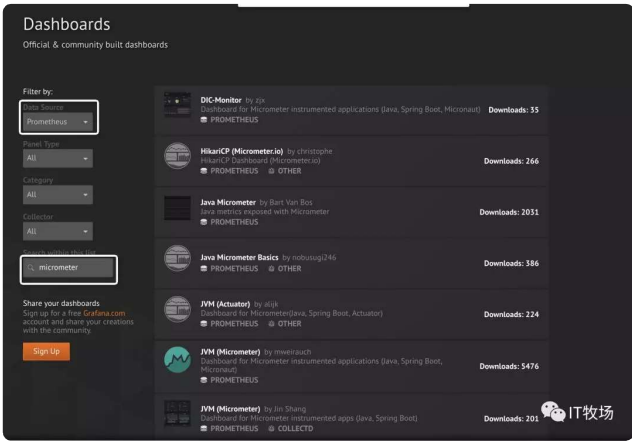
4 按下图步骤操作，即可为Dashboard添加新的图表。



Dashboard市场

至此，已可用Grafana实现较为丰富的图表展示——可将关心的监控指标放置到Dashboard上，并且非常灵活！然而，这个配置的操作虽然不难，但还是挺费时间的.....

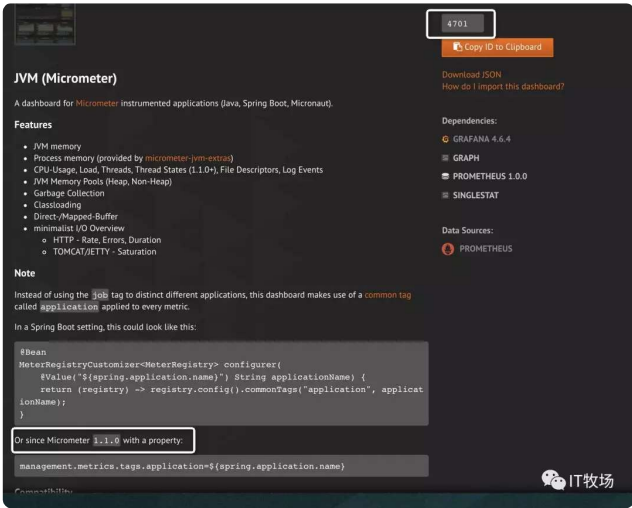
那么是否有配置好的又强大、又通用、拿来即用的Dashboard呢？答案是肯定的！前往 [Grafana Lab - Dashboards^{\[25\]}](#)，输入关键词即可搜索指定Dashboard。



如上图所示，可以找到若干款以 **Prometheus** 作为数据源，支持Micrometer的Dashboard！下面，来演示如何使用 **JVM(Micrometer)** 这个Dashboard。

其他Dashboard使用方式大同小异，建议使用前先点击了解下详情，一般Dashboard中都会有说明。

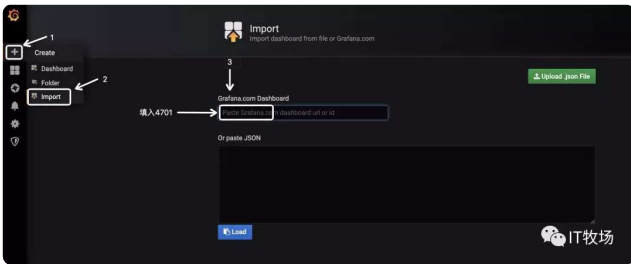
1 点击 **JVM(Micrometer)** 进入Dashboard详情介绍页，如下图所示：



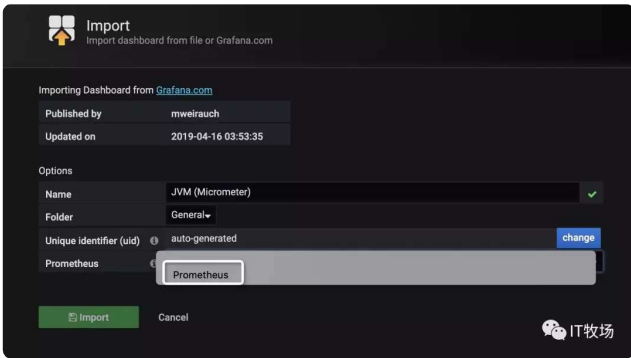
如图已详细描述了该Dashboard的特性、配置。其中的 **management.metrics.tags.application**，前面已经配置过了；本例也不打算演示 **Process memory** 特性，所以 **micrometer-jvm-extras** 怎么用也不打算引入了(这是该Dashboard作者为Micrometer提供的增强包，有兴趣的童鞋可自行点击过去研究)。

细心的童鞋会发现，该页的右上角有个 **4701** 这是一个非常重要的数字！

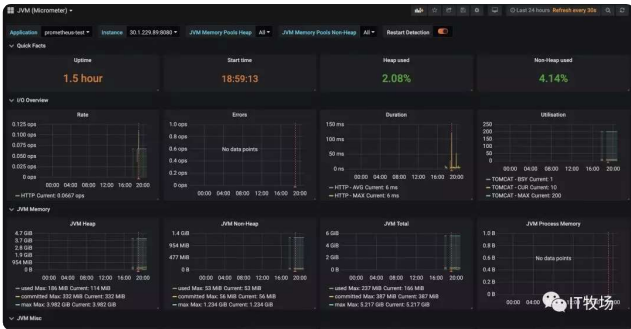
2 访问 **http://localhost:3000**，按下图步骤操作：



3 输入后即可看到类似如下的界面，选择数据源，并点击Import。



4 此时，即可看到类似如下的界面，如图所示，我们常关心的指标该Dashboard均已支持！



比较好用的Dashboard

- [JVM \(Micrometer\)](#)^[26]
- [JVM \(Actuator\)](#)^[27]
- [Spring Boot Statistics](#)^[28]，注：该Dashboard可能有Bug，笔者未能直接测试成功，做了一些改动后才能正常显示。

以上三款是笔者亲测较为全面、好用的Dashboard，仅供参考。

告警

Grafana支持的告警渠道非常丰富，例如邮件、钉钉、Slack、Webhook等，非常实用！比较简单，各位看官自行研究下吧。

配套代码

- [GitHub^{\[29\]}](#)
- [Gitee^{\[30\]}](#)

其中 `prometheus+grafana` 目录已提供Docker一键部署Prometheus + Grafana的Shell

References

- [1] AppOptics: <https://docs.spring.io/spring-boot/docs/2.1.4.RELEASE/reference/htmlsingle/#production-ready-metrics-export-appoptics>
- [2] Atlas: <https://docs.spring.io/spring-boot/docs/2.1.4.RELEASE/reference/htmlsingle/#production-ready-metrics-export-atlas>
- [3] Datadog: <https://docs.spring.io/spring-boot/docs/2.1.4.RELEASE/reference/htmlsingle/#production-ready-metrics-export-datadog>
- [4] Dynatrace: <https://docs.spring.io/spring-boot/docs/2.1.4.RELEASE/reference/htmlsingle/#production-ready-metrics-export-dynatrace>
- [5] Elastic: <https://docs.spring.io/spring-boot/docs/2.1.4.RELEASE/reference/htmlsingle/#production-ready-metrics-export-elastic>
- [6] Ganglia: <https://docs.spring.io/spring-boot/docs/2.1.4.RELEASE/reference/htmlsingle/#production-ready-metrics-export-ganglia>
- [7] Graphite: <https://docs.spring.io/spring-boot/docs/2.1.4.RELEASE/reference/htmlsingle/#production-ready-metrics-export-graphite>
- [8] Humio: <https://docs.spring.io/spring-boot/docs/2.1.4.RELEASE/reference/htmlsingle/#production-ready-metrics-export-humio>
- [9] Influx: <https://docs.spring.io/spring-boot/docs/2.1.4.RELEASE/reference/htmlsingle/#production-ready-metrics-export-influx>
- [10] JMX: <https://docs.spring.io/spring-boot/docs/2.1.4.RELEASE/reference/htmlsingle/#production-ready-metrics-export-jmx>
- [11] KairosDB: <https://docs.spring.io/spring-boot/docs/2.1.4.RELEASE/reference/htmlsingle/#production-ready-metrics-export-kairos>
- [12] New Relic: <https://docs.spring.io/spring-boot/docs/2.1.4.RELEASE/reference/htmlsingle/#production-ready-metrics-export-newrelic>
- [13] Prometheus: <https://docs.spring.io/spring-boot/docs/2.1.4.RELEASE/reference/htmlsingle/#production-ready-metrics-export-prometheus>
- [14] SignalFx: <https://docs.spring.io/spring-boot/docs/2.1.4.RELEASE/reference/htmlsingle/#production-ready-metrics-export-signalfx>
- [15] Simple (in-memory): <https://docs.spring.io/spring-boot/docs/2.1.4.RELEASE/reference/htmlsingle/#production-ready-metrics-export-simple>
- [16] StatsD: <https://docs.spring.io/spring-boot/docs/2.1.4.RELEASE/reference/htmlsingle/#production-ready-metrics-export-statsd>
- [17] Wavefront: <https://docs.spring.io/spring-boot/docs/2.1.4.RELEASE/reference/htmlsingle/#production-ready-metrics-export-wavefront>
- [18] Prometheus官方网站: <https://prometheus.io/>
- [19] Prometheus GitHub: <https://github.com/prometheus>
- [20] Prometheus安装说明: <https://prometheus.io/docs/prometheus/latest/installation/>

[21] Prometheus Configuration官方文档:

档: <https://prometheus.io/docs/prometheus/latest/configuration/configuration/>

[22] Grafana官方网站: <https://grafana.com/>

[23] Grafana GitHub: <https://github.com/grafana/grafana>

[24] Grafana安装说明: <https://grafana.com/docs/installation/>

[25] Grafana Lab - Dashboards: <https://grafana.com/dashboards>

[26] JVM (Micrometer): <https://grafana.com/dashboards/4701>

[27] JVM (Actuator): <https://grafana.com/dashboards/9568>

[28] Spring Boot Statistics: <https://grafana.com/dashboards/6756>

[29] GitHub: <https://github.com/eacdy/spring-boot-study/tree/master/spring-boot-actuator-prometheus-grafana>

[30] Gitee: <https://gitee.com/itmuch/spring-boot-study/tree/master/spring-boot-actuator-prometheus-grafana>

文章转载自公众号

 IT牧场 >